xTB Backends for ChemDM

dxtb on GPU, and the `xtb-python` → `tblite` Migration

Notes for ChemDM

July 2026

Abstract

We set out to accelerate ChemDM's semi-empirical potential by moving xTB onto the GPU with **dxtb** (a differentiable PyTorch reimplementation). Two findings came out, neither the one we went looking for. First, **dxtb on GPU is not faster than CPU-multiprocessed xtb** for our workloads: across a full size × batch sweep a single CUDA GPU never beat a 12-core `xtb` pool, best case still 3.3× slower — for architectural reasons, not tuning. Second, the production backend `xtb-python` is **deprecated and has a genuine gradient bug** (analytic force ≠ finite-difference gradient of its own energy by ∼0.22–0.24 eV/Å at symmetric geometries), present in *both* GFN1 and GFN2 — and GFN2 is our production default. The fix is to migrate the backend to **tblite**, which we verified reproduces `xtb-python` to within SCF noise, is free of the gradient bug, *and* is faster on CPU (up to ∼4× for large molecules). **Recommendation: keep CPU-multiprocessed evaluation, swap the backend `xtb-python` → tblite.** Everything here reproduces from `examples/xtb_gpu/`.

1 Why we cared about dxtb

ChemDM's workhorse potential is `XTBPotential` (`src/chemdm/xtbSetup.py`), a thin ASE wrapper around xTB, called constantly and in bulk: transition-path (NEB) calculations evaluate every image of a band each optimizer step, and conformer generation relaxes hundreds of conformers of the *same* molecule. Both have a GPU-friendly shape — *large, homogeneous batches*: a NEB band is N copies of the same atoms at different geometries; a conformer set is N copies of one molecule. Same atomic numbers, zero padding, identical basis size.

dxtb (grimme-lab) is a fully differentiable, pure-PyTorch implementation of the xTB Hamiltonians, promising three things: (i) **GPU execution** of the SCF; (ii) **batching** a whole band / conformer set into one stacked call with per-system convergence masking; (iii) **autodiff forces**, making the potential itself differentiable (for training against xTB, sensitivity analysis, end-to-end optimization). The plan: validate GFN1 locally (no CUDA), then GFN2 + speed on the Linux cluster.

2 dxtb is correct — and it exposed a bug in `xtb-python`

Stage 1 (GFN1, CPU, float64) over a 10-molecule suite (H_2 up to a 182-atom C60-alkane) gave essentially exact correspondence with the production potential: energy error mean $\sim 2 \times 10^{-6}$ Ha (worst 1.1×10^{-5} Ha at 182 atoms), force max error $\leq 1.8 \times 10^{-5}$ eV/Å, cosine 1.0. dxtb is a faithful reimplementation. But used as an independent reference it *disagreed* with `xtb-python` by ~ 0.2 eV/Å on one atom at the methanol equilibrium — far above the $\sim 10^{-5}$ agreement everywhere else. That kicked off the second thread (§4); the punchline is that `xtb-python` was the one that was wrong.

3 The speed question: dxtb-GPU vs multiprocessed xtb

This is the measurement the GPU idea rested on. Run on the Linux cluster, GFN2-xTB, energy + forces, min over 3 reps, comparing batched dxtb (`cuda:0`, float64) against the incumbent `xtb-mp` — ChemDM's spawn `ProcessPool` over **12 CPU processes** — with `xtb-seq` (single process) and dxtb on CPU for context. Time is per batch of size B (lower is better at fixed B); construction/startup/warm-up excluded.

3.1 Crossover summary (dxtb-cuda vs xtb-mp, 12 procs)

molecule	n	smallest B where dxtb-GPU wins	peak speedup
thiophene	9	never	0.08×
benzene	12	never	0.10×
caffeine	24	never	0.12×
cholesterol	74	never	0.30×

dxtb on a CUDA GPU never beats the 12-core xtb pool — anywhere in the grid, for any batch size up to 256. Its best showing (cholesterol) is still 3.3× slower.

3.2 Full 2-D sweep (ms per batch of size B)

	thiophene ($n = 9$)				**benzene** ($n = 12$)			
B	xtb-mp	xtb-seq	dxtb-cu	dxtb-cpu	xtb-mp	xtb-seq	dxtb-cu	dxtb-cpu
1	5.2	888.9	234.9	150.1	5.1	6.8	236.7	142.9
2	6.5	1309.8	244.8	182.9	6.5	13.3	241.5	166.3
4	6.0	2475.4	259.1	207.9	5.9	26.4	254.9	198.9
8	5.1	5227.4	272.8	267.9	5.8	53.6	279.4	305.7
16	10.0	11611.7	310.5	444.5	10.8	104.4	322.8	399.9
32	19.3	23900.8	416.1	764.4	16.2	208.8	402.3	701.6
64	33.1	51414.9	555.0	973.9	32.1	410.1	588.2	1170.2
128	56.3	102243.0	898.5	1663.2	94.7	819.1	904.3	1872.4
256	118.5	191643.9	1575.2	3162.4	114.9	1714.8	1594.5	3291.7

	caffeine ($n = 24$)				**cholesterol** ($n = 74$)			
B	xtb-mp	xtb-seq	dxtb-cu	dxtb-cpu	xtb-mp	xtb-seq	dxtb-cu	dxtb-cpu
1	17.2	23.2	266.0	199.7	88.8	82.0	298.1	539.8
2	18.2	44.1	294.1	296.2	88.7	163.9	397.0	840.4
4	17.1	77.4	306.8	394.6	88.8	325.9	469.0	1251.9
8	16.7	154.7	376.4	618.9	90.7	648.1	603.3	1819.5
16	33.5	314.1	447.5	849.2	175.7	1290.0	797.9	3269.2
32	49.4	626.6	608.7	1402.5	264.1	2575.2	1261.6	5599.8
64	100.3	1260.9	910.8	2542.7	522.5	5406.0	2262.3	10741.3
128	192.9	2478.7	1631.8	4307.4	972.3	10712.9	4604.8	20378.7
256	373.4	4938.3	3315.1	8415.6	1925.6	21459.3	11101.4	43307.1

(The `xtb-seq` thiophene column is inflated by a first-molecule import/warm-up artifact — it is the first molecule in the sweep; not the baseline, and it changes no conclusion.)

3.3 Reading the two trends

The data is a clean, consistent story that matches the dxtb source:

- **Bigger molecules help.** Peak speedup climbs $0.08 \to 0.10 \to 0.12 \to 0.30\times$ across $9 \to 12 \to 24 \to 74$ atoms — the only genuinely GPU-batched part (the SCF eigensolve, $O(n_{\text{ao}}^3)$) growing as a fraction of total work. Extrapolated crossover to parity is far off (several hundred atoms), right where forces (autograd graph, memory $\sim B\, n_{\text{ao}}^2$) begin to OOM.
- **Bigger batches do *not* help large molecules.** For cholesterol the per-B speedup *peaks at* $B = 1$ ($0.30\times$) and *degrades* to $0.17\times$ at $B = 256$. That is the tell: GFN2's integrals go through `libcint`, which runs on CPU, serially, one molecule at a time, so the integral cost grows ∼linearly in B and swamps the batched SCF gain — while `xtb-mp` parallelizes those integrals across 12 cores. For *small* molecules batching does help relatively (thiophene $0.02 \to 0.08\times$) because the ∼235 ms fixed GPU/launch floor gets amortized.

Root cause (confirmed from dxtb 0.4.0 source). dxtb's GPU path is a *hybrid*. The SCF is genuinely batched on device (one batched `torch.linalg.eigh` on $[B, n_{\text{ao}}, n_{\text{ao}}]$ per step, batched Fock/density/Fermi, per-system convergence masking + culling). But GFN2 integrals (and derivatives) go through `libcint` on CPU, looped per molecule, with a CPU↔GPU transfer each step. Only a fraction is accelerated, and the serial CPU floor grows with B. This is architectural, not a knob. (GFN1 has a pure-torch driver, `int_driver="autograd"`, that keeps everything on the GPU — the one untested path with a real shot at a win.)

3.4 The CPU picture: batching is the wrong instinct there

An earlier local benchmark (aspirin $\times N$, GFN1, energy + forces, CPU) at $N = 128$:

mode	time
`xtb-mp` (ProcessPool, spawn, all cores)	**340 ms**
`xtb-seq` (single process)	1120 ms
`dxtb-mp` (dxtb non-batched across procs)	1710 ms
`dxtb-cpu-batched` (one big batched call)	10044 ms

On CPU, dxtb loses every way, and **batching it is $\sim 6\times$ *worse* than parallelizing it** (one giant single-threaded autograd graph + one huge `eigh` vs. many small ones across cores). Batching pays only when the linear algebra is massively parallel, i.e. on a real GPU — which §3.1 then showed still isn't enough for GFN2.

3.5 What dxtb *is* still good for

The one thing `xtb-mp` cannot do is give gradients *through* the potential. dxtb's autodiff forces make the whole potential differentiable — the real prize for training a model against xTB or end-to-end optimization. That is a capability advantage, not a speed one, and it carries a real runtime cost (the retained autograd graph). Parked as a later, separate goal.

Speed verdict: keep CPU-multiprocessed xtb for GFN1 and GFN2 evaluation.

4 The xtb-python gradient bug

Staying on CPU xtb raises a correctness question about the *specific* backend. `XTBPotential` uses `xtb-python`, which is **deprecated** (confirmed by the grimme-lab maintainer) — and has a real gradient bug.

What it is. At the C_s-symmetric equilibrium of methanol (the ASE G2 geometry: four atoms in a mirror plane, two mirror-image methyl H), `xtb-python`'s *analytic force disagrees with the finite-difference gradient of its own energy* — a self-contradiction needing no external reference. The neutral arbiter is −FD(own energy); dxtb (autograd) and tblite both match their own FD to the FD floor, `xtb-python` does not.

It is geometry-specific. The defect fires at *symmetric* geometries, not at generic ones. This is important for reading the next section: the correspondence suite (§5) contains a molecule also called "methanol", but at an *asymmetric* RDKit/ETKDG conformer, where `xtb-python` is perfectly self-consistent and agrees with tblite to 4.3×10^{-5} eV/Å. So the same molecule looks fine there and broken here — because it is two different geometries, and only the symmetric one triggers the bug. This is exactly why a correspondence sweep over ordinary conformers never surfaces the defect, and likely why it went unreported for so long.

GFN1. `xtb-python` self-consistency error = 2.2517×10^{-1} eV/Å (worst atom C, only the x-component wrong); tblite = 7.4073×10^{-5}. The error is *flat across SCF accuracy* $1.0 \to 10^{-4}$, ruling out convergence / smearing / step-size. tblite-vs-dxtb agree to 7.5×10^{-5}.

GFN2 — the production method.

self-consistency: $\max \lvert F_{\text{analytic}} - (-\text{FD}) \rvert$	GFN1	GFN2	
`xtb-python`	2.2517×10^{-1}	2.3626×10^{-1}	(worst: O)
tblite	7.4073×10^{-5}		1.2539×10^{-4}

The defect is present in **GFN2 too** (0.236 eV/Å, again accuracy-invariant, again one Cartesian component of one heavy atom). The worst atom shifts (C for GFN1, O for GFN2) but magnitude/character match — strong evidence GFN1 and GFN2 *share the buggy gradient code path* (a geometry-only, non-self-consistent term, since it is accuracy-invariant; not the method-specific dispersion/multipole parts). **Because `XTBPotential` defaults to GFN2, this is affecting production forces now.** The bug was reported and the maintainer confirmed it is an `xtb-python` bug (not dxtb), not previously known; tblite is self-consistent for both methods, i.e. migrating fixes it.

5 tblite: correspondence verified

tblite (grimme-lab) is the maintained xTB library the maintainer recommends over `xtb-python`. `TBLitePotential` (`examples/xtb_gpu/tblite_potential.py`) is a drop-in subclass of `XTBPotential`: it overrides only the calculator construction and inherits `energy_forces`, keeping the parent's (`charge`, `uhf`) signature and bridging tblite's `multiplicity` = `uhf`+1 internally. `check_correspondence.py` compares it against the `xtb-python` reference over the suite.

		GFN1				
molecule	n	ΔE[Ha]	$F_{\max}$	F_{rmse}	$F_{\cos}$	ΔE[Ha]
H_2	2	-3.82×10^{-10}	2.37×10^{-7}	1.37×10^{-7}	1.0000000	-1.49×10
H_2O	3	-4.20×10^{-9}	3.57×10^{-5}	1.55×10^{-5}	1.0000000	-2.59×10
methanol	6	-5.77×10^{-9}	4.32×10^{-5}	1.61×10^{-5}	0.9999999	-3.70×10
thiophene	9	-1.07×10^{-8}	2.44×10^{-5}	8.61×10^{-6}	1.0000000	-6.30×10
benzene	12	-9.73×10^{-9}	2.11×10^{-6}	9.72×10^{-7}	1.0000000	-9.38×10
aspirin	21	-2.75×10^{-8}	5.61×10^{-5}	1.21×10^{-5}	1.0000000	-2.29×10
caffeine	24	-3.32×10^{-8}	7.07×10^{-5}	1.92×10^{-5}	0.9999999	-3.08×10
cholesterol	74	-4.53×10^{-8}	3.28×10^{-5}	3.74×10^{-6}	1.0000000	-5.60×10
atorvastatin	76	-9.24×10^{-10}	1.05×10^{-4}	1.43×10^{-5}	0.9999999	1.28×10
C60-alkane	182	4.18×10^{-6}	1.17×10^{-5}	3.05×10^{-6}	0.9999999	6.14×10

Forces in eV/Å. GFN1 aggregate: $|\Delta E|$ mean 4.3×10^{-7} Ha, $F_{\max}$ mean 3.8×10^{-5} (max 1.0×10^{-4}), worst cos = 0.999999996. GFN2 aggregate: $|\Delta E|$ mean 7.4×10^{-6} Ha, $F_{\max}$ mean 2.0×10^{-4} (max 7.7×10^{-4}), worst cos = 0.999999813.

The suite geometries are generic RDKit/ETKDG conformers — *asymmetric* — so `xtb-python`'s symmetric-geometry gradient bug (§4) does not fire here; that is why its methanol row shows tiny errors ($F_{\max} = 4.3 \times 10^{-5}$ eV/Å GFN1) even though the *same molecule* at its symmetric equilibrium is 0.23 off in §4. Same molecule, different geometry.

Both methods correspond to within SCF/backend noise. GFN2's forces diverge a little more (multipole + self-consistent D4 gradients) and the divergence *grows with size* — accumulating numerical noise, not a defect. Indeed tblite's GFN2 analytic force matches the FD of its *own* energy to 1.25×10^{-4} eV/Å, i.e. tblite is self-consistent (§4); the tblite-vs-xtb difference is `xtb-python` noise/bug, not tblite error.

6 xtb-python vs tblite: CPU timing

Since evaluation stays CPU-multiprocessed, the decision-relevant number is per-call latency at one thread (production runs single-threaded pool workers). Each backend timed in its own subprocess, OMP/BLAS pinned to 1, best of 10, warm-up excluded, distinct geometry per rep (ASE caches identical geometries).

		GFN2 (production)			**GFN1**		
molecule	n	`xtb-python`	tblite	ratio	`xtb-python`	tblite	ratio
H_2	2	0.15	0.25	1.72×	0.17	0.22	1.29×
H_2O	3	0.19	0.26	1.40×	0.29	0.31	1.08×
methanol	6	0.37	0.43	1.15×	0.82	0.65	0.79×
thiophene	9	1.47	1.26	0.86×	2.66	2.05	0.77×
benzene	12	1.64	1.34	0.81×	3.44	2.35	0.68×
aspirin	21	6.31	4.77	0.76×	11.55	7.20	0.62×
caffeine	24	8.43	5.63	0.67×	14.78	8.83	0.60×
cholesterol	74	63.37	37.05	0.58×	126.48	53.15	0.42×
atorvastatin	76	98.75	50.97	0.52×	171.35	66.32	0.39×
C60-alkane	182	470.96	144.46	0.31×	996.07	226.04	0.23×

Per-call wall time in ms; ratio = tblite/`xtb-python` (< 1 ⇒ tblite faster). Geometric-mean ratio: GFN2 0.79×, GFN1 0.62×.

tblite is faster on CPU, and the advantage widens sharply with molecule size. Sub-10-atom molecules are a wash (sub-millisecond either way, tblite marginally slower from fixed overhead), but from thiophene up tblite pulls ahead, reaching 3.2× (GFN2) and 4.3× (GFN1) on the 182-atom C60-alkane — exactly the regime where evaluation cost actually matters (NEB on large substrates, big conformers). So the migration is not a speed sacrifice for correctness; it is a speed *win*.

7 Engineering notes: OpenMP and how the harness runs

Getting the verification to run was its own adventure, worth recording.

- **tblite install.** Needs the Python bindings (`tblite-python` on conda-forge, or `pip install tblite`), not the bare `tblite` C library. The conda-forge `tblite 0.6.0` osx-arm64 build has a broken GFN2 path (segfaults for any GFN2 input); the PyPI `tblite 0.7.0` wheel fixes it. GFN2 was never inherently "Linux-only" for tblite — that is real only for *dxtb* (needs Linux-only `tad-libcint`).
- **The OpenMP clash.** The pip tblite wheel *vendors its own* `libomp`. Running `xtb-python` (conda OpenMP) and tblite (vendored) in one process gives `OMP: Error #15` → abort; `KMP_DUPLICATE_LIB_OK` escalates to segfault; `multiprocessing.spawn` deadlocks. Removing openmm from the import graph did *not* fix it — the clash is two `libomp` copies. conda-forge tblite (shared OpenMP) would fix it but is blocked by the `openmm-xtb` package pinning py3.13/3.14 vs. our py3.11.
- **The fix: one backend per process.** Both harness scripts delegate each evaluation to `subproc_worker.py` via `subprocess.run` — a fresh interpreter importing exactly one backend. This mirrors production (xtb already runs in a spawn `ProcessPool`, one backend per worker), so **the clash never arises in production**; it is purely a harness artifact.
- **Cleanups.** `create_xtb_system`/`create_xtb_context` (the only openmm-dependent helpers) moved out of `xtbSetup.py` into `src/chemdm/openmmSetup.py`, so importing `XTBPotential` no longer pulls openmm; the three example callers were repointed.

8 Recommendations and open items

Recommendations.

1. **Evaluation stays CPU-multiprocessed xtb.** dxtb-GPU does not beat a 12-core pool for GFN2 at our sizes; the bottleneck (serial CPU `libcint`) is architectural.
2. **Migrate the backend `xtb-python` → tblite.** It matches to backend noise, is maintained, fixes a gradient bug currently affecting production GFN2 forces, *and* is ∼1.3–4× faster on CPU for non-trivial molecules.
3. **Keep dxtb only for differentiability.** Autodiff forces are its unique capability; revisit when a differentiable-potential use case is on the table.

Open items.

- The production swap itself (`XTBPotential` → tblite): replace outright vs. a selectable `backend=` parameter. To be proposed.
- GFN1 `int_driver="autograd"` CUDA sweep — the one dxtb config that keeps integrals on-GPU, the only remaining path that could show a GPU win.
- Full 3-way (incl. dxtb) correspondence on the Linux cluster.

Reproduce: from `examples/xtb_gpu/`, run `check_correspondence.py`, `reference_gradient_bug.py` (both with a `METHOD` constant), and `benchmark_backends.py`. On the cluster `RUN_DXTB` auto-enables and the comparisons become three-way.